

1 LSTM 模型用于非线性补偿的多震级训练

1.1 1. 背景

动圈检波器在高频区域常出现假频非线性失真问题，这种失真主要表现为信号在频域上产生不应有的频率分量，严重影响传感器的测量精度。LSTM (Long Short-Term Memory) 神经网络因其能有效建模时序数据的非线性特性，成为解决此类问题的有力工具。本文档详细描述了基于 LSTM 的多阶段增量学习策略，用于动圈检波器假频非线性补偿。

1.1.1 数据来源

本研究的数据来自使用 5Hz 动圈检波器在振动台上进行的扫频实验。原始数据包括多个震级水平下的时序数据，具体文件如下：

2025/03/30	16:41	254,985	output_20241217_212141_VAS-300-03_A16_震级 2.0_analyze.json
2025/03/30	16:40	3,197,509,304	output_20241217_212141_VAS-300-03_A16_震级 2.0_data.json
2025/03/30	16:44	254,816	output_20241218_094245_VAS-300-03_A16_震级 3.0_analyze.json
2025/03/30	16:43	3,197,385,224	output_20241218_094245_VAS-300-03_A16_震级 3.0_data.json
2025/03/30	16:48	254,712	output_20241218_152549_VAS-300-03_A16_震级 4.0_analyze.json
2025/03/30	16:47	3,197,285,180	output_20241218_152549_VAS-300-03_A16_震级 4.0_data.json
2025/03/30	16:53	254,712	output_20241218_183659_VAS-300-03_A16_震级 5.0_analyze.json
2025/03/30	16:51	3,197,409,296	output_20241218_183659_VAS-300-03_A16_震级 5.0_data.json
2025/03/07	21:45	255,407	output_20241224_110004_VAS-300-03_A16_震级 1.0_analyze.json
2025/03/07	21:44	3,197,585,288	output_20241224_110004_VAS-300-03_A16_震级 1.0_data.json

每个震级水平的数据由一对文件组成：频率特性分析文件(analyze. json)和原始时序数据文件(data. json)。数据涵盖了从 1.0 到 5.0 的五个震级水平，提供了全面的非线性特性分析基础。

1.2 2. LSTM 网络设计

LSTM 模型主要由三个关键组件构成：

LSTM 模型 → 全连接层 → 输出层

这种架构能够有效捕获时间序列中的长期依赖关系，并通过全连接层进行特征转换，最终生成补偿后的输出信号。

1.2.1 LSTM 层配置

LSTM 层是模型的核心，负责学习时序数据中的非线性特征：

```
# LSTM 层关键参数
lstm_layer = tf.keras.layers.LSTM(
    units=32,                # 单元数量决定特征提取能力
    activation='tanh',       # 激活函数
    dropout=0.2,             # dropout 比率, 防止过拟合
    return_sequences=True    # 保留时间步信息, 确保时序连续性
)
```

LSTM 层的单元数设置为 32，这是在特征提取能力和计算效率间的平衡选择。return_sequences=True 确保输出保留完整的时间序列信息，为后续处理提供基础。

1.2.2 全连接层配置

全连接层负责进一步处理 LSTM 提取的特征：

```
# 全连接层配置
dense_layer = tf.keras.layers.Dense(
    units=32,          # 与LSTM层保持相同单元数
    activation='relu'  # ReLU 激活函数有助于引入非线性特性
)
```

全连接层与 LSTM 层保持相同的单元数(32)，通过 ReLU 激活函数引入额外的非线性变换，增强模型对复杂特征的学习能力。

1.2.3 输出层配置

输出层负责生成最终预测结果：

```
# 输出层配置
output_layer = tf.keras.layers.Dense(
    units=1,          # 单一输出通道
    activation=None    # 线性输出，无激活函数
)
```

输出层使用线性激活函数，确保模型能够生成连续的预测值，这对于波形补偿任务至关重要。

完整的 LSTM 模型实现如下，采用面向对象的方式设计模型类：

```
class LSTM:
    def __init__(self,
                  lstm_units=32,
                  lstm_dropout=0.2,
                  activation='tanh',
                  fs=2000,
                  checkpoint_dir='data',
                  model_subcfg={}):

        # 初始化模型名称
        self.model_name = 'LSTM'

        # 构建序列模型结构
        self.model = tf.keras.Sequential()
        self.model.add(tf.keras.layers.LSTM(
            units=lstm_units,
            activation=activation,
            dropout=lstm_dropout,
            return_sequences=True
        ))

        # 全连接层
        self.model.add(tf.keras.layers.Dense(32, activation='relu'))

        # 输出层
        self.model.add(tf.keras.layers.Dense(1))

        # 设置输入形状 (None, None, 1)
        self.model.build(input_shape=(None, None, 1))

        # 补充注释
        # 输入维度说明: (batch_size, time_steps, features)
        # 示例: (None, 8000, 1) 表示任意批量×4 秒数据×单通道
```

输入形状设置为 (None, None, 1)，表示任意批量大小和序列长度的单通道时序数据。

1.2.4 核心方法实现

下面是模型的核心方法实现，包括编译、训练和权重加载等功能：

```
def compile(self, *args, **kwargs):
    """编译模型，设置损失函数和优化器"""
    self.model.compile(*args, **kwargs)

def fit(self, *args, **kwargs):
    """训练模型，支持回调函数"""
    # 添加回调函数列表
    if 'callbacks' not in kwargs:
        kwargs['callbacks'] = []

    history = self.model.fit(*args, **kwargs)
    return history

def load_weights(self, filepath):
    """加载模型权重并保存权重JSON描述"""
    self.model.load_weights(filepath)

def save_weights(self, filepath):
    """保存模型权重并生成权重JSON描述"""
    self.model.save_weights(filepath)
```

这些方法实现了模型的编译、训练和权重管理功能。

1.3 3. 数据预处理

数据集由多个震级和频率条件下的传感器响应组成，具有以下特性：

```
# 数据集关键参数
sampling_rate = 2000 # Hz
time_window = 4.0 # 秒
data_shape = (magn_num, freq_num, points_num)
# 其中:
# - magn_num: 震级数量
# - freq_num: 频率数量
# - points_num: 每个样本的时间点数量 = sampling_rate * time_window
```

数据采集系统设置为 2000 Hz 采样率，每个样本包含 4 秒的时间窗口。数据组织为三维张量，便于按震级和频率进行批处理和分析。

数据预处理包括特征标准化和缓存加速：

```
# 特征处理核心逻辑
dataset_type = 'Alias' # 代表假频失真
feature_range = [-1, 1] # 数据归一化范围
use_cache = True # 启用特征缓存加速训练
```

使用 Alias 类型数据专门针对假频失真问题，数据标准化到 [-1, 1] 范围内，这有助于模型训练的稳定性。

1.3.1 自定义缩放器实现

数据缩放是预处理的关键步骤，为此我们实现了一个自定义的缩放器类：

```
class CustomScaler:
    def __init__(self, feature_range=(0, 1)):
        self.feature_range = feature_range
        self.data_min_ = None
        self.data_range_ = None

    def fit(self, X):
        # 计算每列的最小值和最大绝对值 (data_min_ 和 data_range_)
        self.data_min_ = 0
        # 将多维数据展开为一维
        X_flattened = X.flatten()
        self.data_range_ = np.max(np.abs(X_flattened))
        self.data_range_ = self.data_range_ / max(abs(self.feature_range[1]),
                                                  abs(self.feature_range[0]))

    def transform(self, X):
        # 计算缩放比例, 先 copy 一份数据
        X = X.copy()
        return X / self.data_range_

    def fit_transform(self, X):
        self.fit(X)
        return self.transform(X)

    def dump_json(self, file_path):
        """
        将缩放器参数保存到 JSON 文件

        Args:
            file_path: JSON 文件的路径
        """
        # 准备要存储的数据
        data = {
            'feature_range': list(self.feature_range),
            'data_min_': float(self.data_min_) if self.data_min_ is not None else None,
            'data_range_': float(self.data_range_) if self.data_range_ is not None else None
        }

        # 确保目录存在
        os.makedirs(os.path.dirname(os.path.abspath(file_path)), exist_ok=True)

        # 保存到文件
        with open(file_path, 'w') as f:
            json.dump(data, f)

    @classmethod
    def from_json(cls, file_path):
        """
        从 JSON 文件加载缩放器参数

        Args:
            file_path: JSON 文件的路径

        Returns:
            CustomScaler: 加载的缩放器实例
        """
```

```

# 从文件加载数据
with open(file_path, 'r') as f:
    data = json.load(f)

# 创建新实例
scaler = cls(feature_range=tuple(data['feature_range']))

# 设置属性
scaler.data_min_ = data['data_min_']
scaler.data_range_ = data['data_range_']

return scaler

```

这个定制化的缩放器不仅支持数据的标准化，还允许通过 JSON 文件保存和加载缩放参数，确保训练和推理阶段使用相同的缩放标准。使用自定义的 CustomScaler 类对数据进行缩放，确保输入特征和目标值都在相同的数值范围内，有助于模型训练的收敛。

1.3.2 将数据重塑为 LSTM 可接受的输入格式

```

# 重塑数据格式以适配 LSTM
# 将(magn_num, freq_num, points_num)变形为(seq_num=magn_num*freq_num, points_num, 1)
X_train = dataset_train.reshape2feature(dataset_train.inputs)
y_train = dataset_train.reshape2feature(dataset_train.output_tar)

```

此步骤将三维数据重塑为 LSTM 期望的输入格式：序列数 $\times$ 时间步 $\times$ 特征数，即(seq_num, points_num, 1)，使模型能够正确处理时序信息。

1.4 4. 多阶段训练策略

我们采用“单震级预训练+多震级增量训练”的分层策略，整体分为四个阶段：

1.4.1 单震级预训练

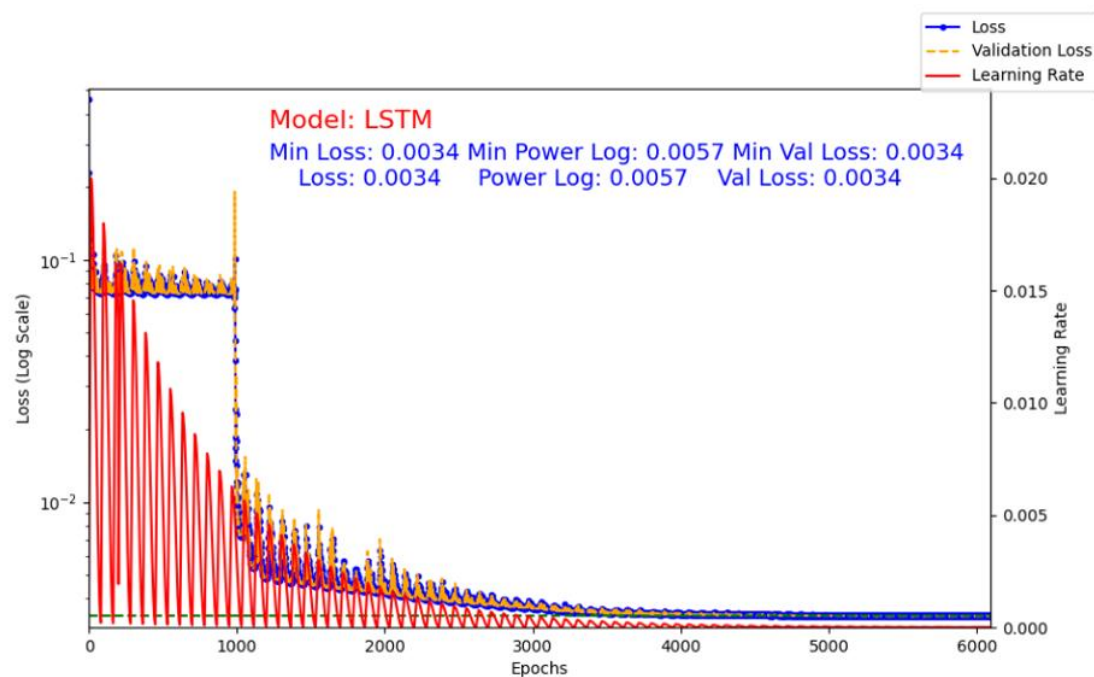
首先在单一震级数据上建立基础模型能力：

```

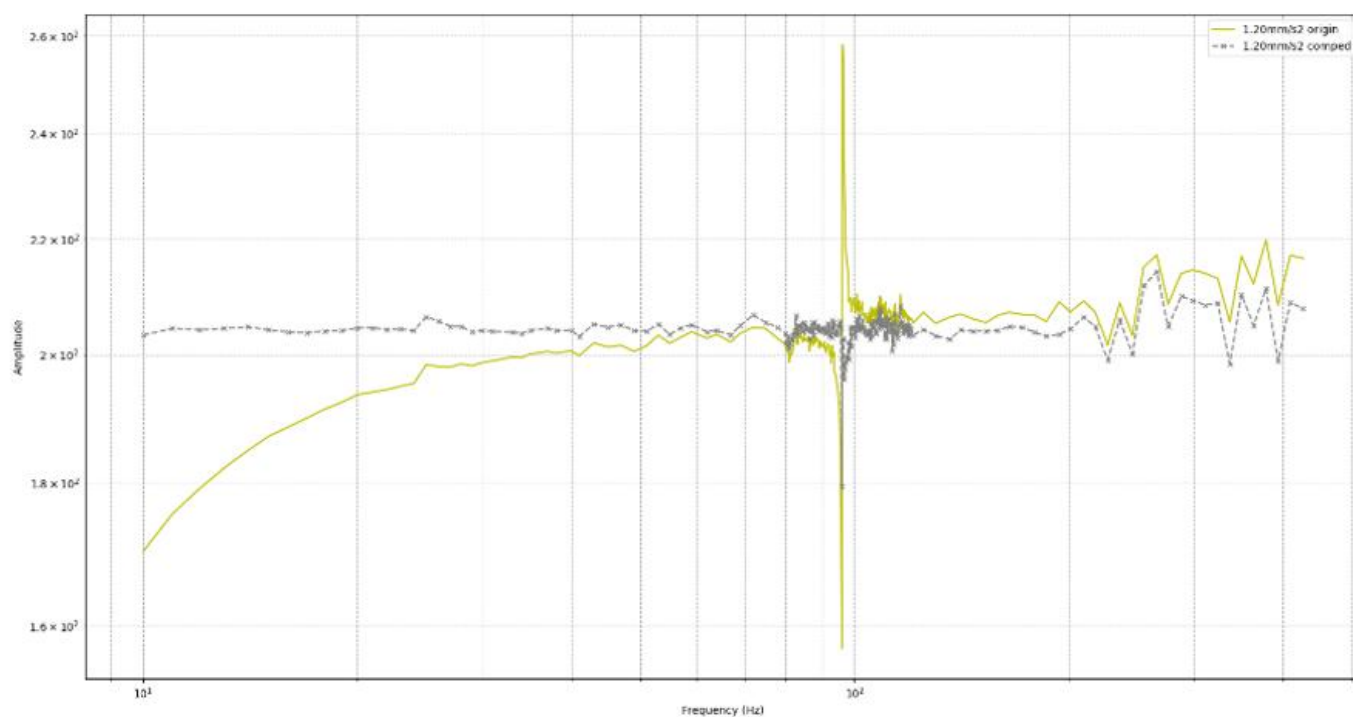
# 单震级预训练配置
config_m5 = {
    "data_path": "data/ALIA_M5", # 单一震级(M5)数据
    "learning_rate": 0.1, # 较高学习率
    "auto_lr_decay_steps": 100, # 快速衰减
    "step_per_epoch": 5, # 每轮执行5个step
    "use_best_val_weights": True # 使用验证集最佳权重
}

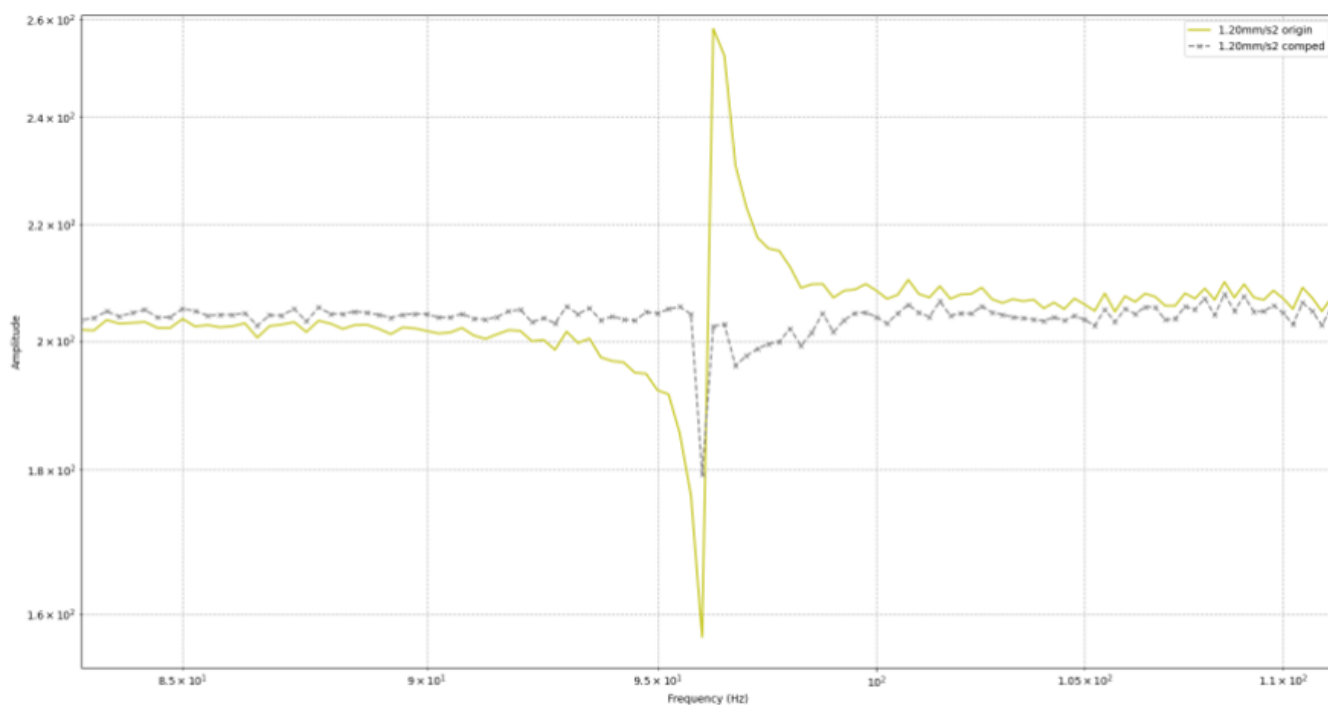
```

单震级预训练在 M5 震级数据上进行，使用较高学习率(0.1)配合快速衰减(每 100 步)，这有助于模型快速找到大致的参数区域。训练结果显示损失值达到 0.00120，为多震级训练奠定了良好基础。



一个震级收敛较好



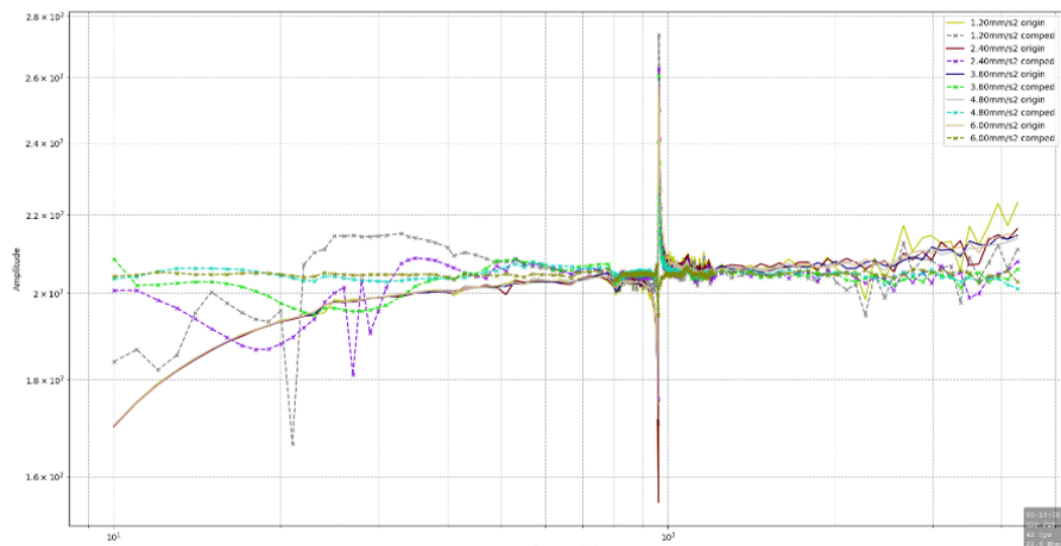


1.4.2 第一阶段增量训练

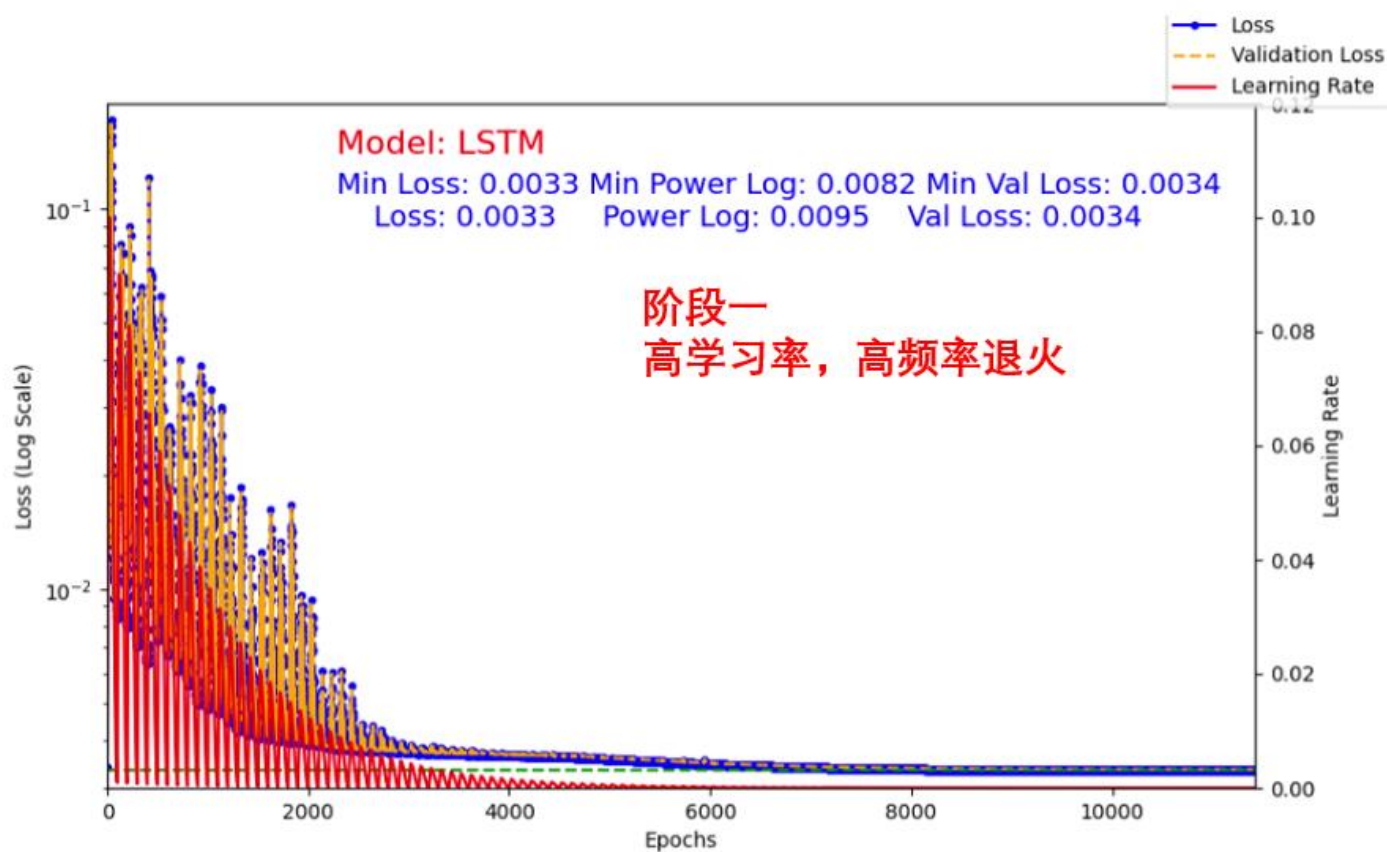
基于单震级预训练模型，进行第一阶段的多震级训练：

```
# 第一阶段训练配置
config_ex1 = {
    "base_project": "LSTMu32a1_rs300_m5", # 继承单震级预训练模型
    "data_path": "data/ALIA_EX",          # 多震级数据
    "learning_rate": 0.1,                  # 保持较高学习率
    "auto_lr_decay_steps": 100,            # 快速衰减
    "step_per_epoch": 1                    # 每轮执行1个step
}
```

第一阶段继承单震级预训练模型的权重，转而在多震级数据上训练。保持较高学习率和快速衰减策略，使模型能够适应更复杂的数据分布。经过 11480 个周期的训练，损失值降至 0.00333。



支持了~~scaler~~继承自基础模型之后，可以对多震级数据正常处理，对训练过的震级表现较好，其他震级一般



1.4.3 第二阶段增量训练

降低学习率，进一步优化模型参数：

```
# 第二阶段训练配置
config_ex2 = {
    "base_project": "LSTMu32a1_rs300_ex", # 继承第一阶段模型
```

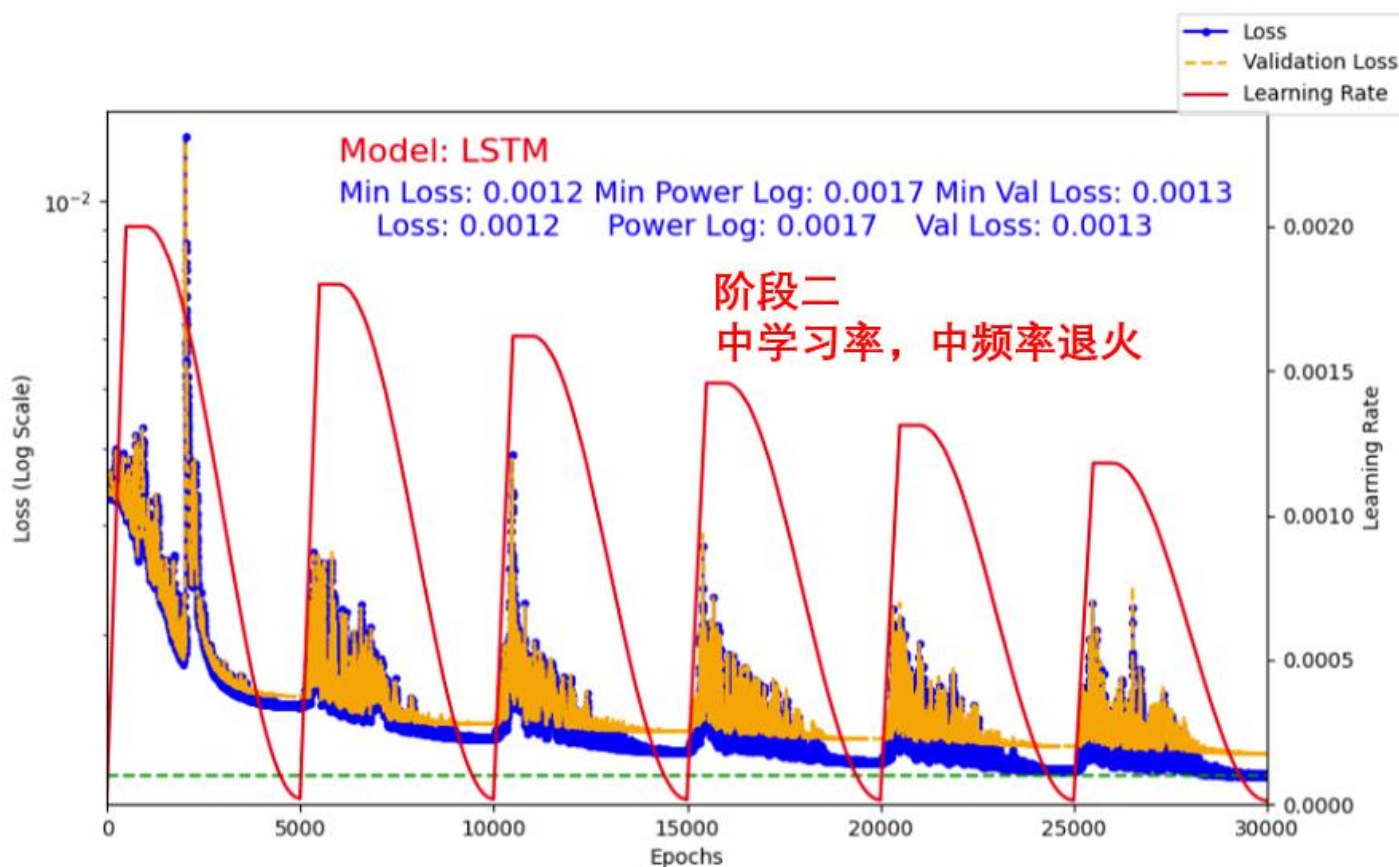


```

"learning_rate": 0.002,          # 降低学习率
"auto_lr_decay_steps": 5000,     # 减缓衰减速度
"step_per_epoch": 1
}

```

第二阶段显著降低了学习率(0.002)并减缓了衰减速度(每 5000 步)，使模型能够进行更精细的参数优化。完成 30000 个周期的训练后，损失值进一步降至 0.00118。



1.4.4 第三阶段增量训练

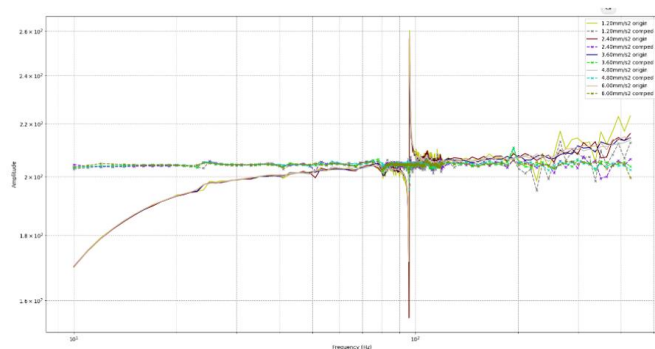
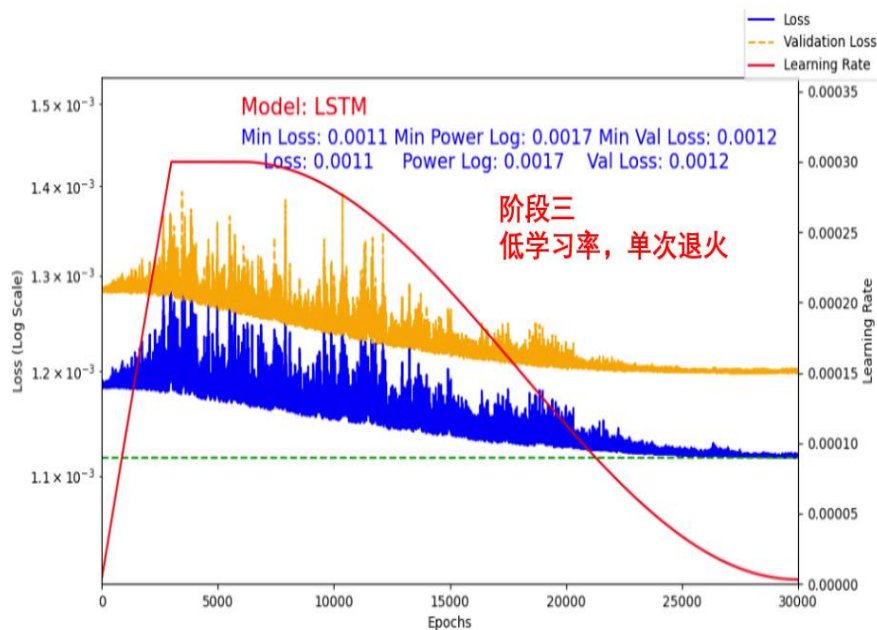
使用更低学习率和更慢衰减，进行最终优化：

```

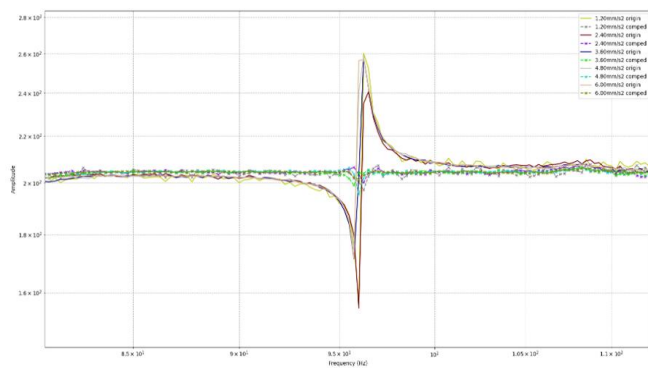
# 第三阶段训练配置
config_ex3 = {
    "base_project": "LSTMu32a1_rs300_ex2", # 继承第二阶段模型
    "learning_rate": 0.0003,               # 更低学习率
    "auto_lr_decay_steps": 30000,          # 极慢衰减
    "step_per_epoch": 1
}

```

第三阶段使用极低的学习率(0.0003)和极慢的衰减速度(每 30000 步)，进行模型的最终微调。经过 30000 个周期的训练，损失值进一步优化至 0.00112，比第二阶段降低约 5.6%。



多阶段训练后的
5Hz检波器补偿
效果



多阶段训练后的
5Hz检波器补偿
效果

1.4.5 各阶段训练状态对比

下表对比了四个阶段的训练成果：

阶段	最小损失	最小验证损失	功率对数损失	验证功率对数损失
单震级预训练	0.00120	0.00128	0.00169	0.00226
第一阶段	0.00333	0.00335	0.00952	0.00957
第二阶段	0.00118	0.00128	0.00175	0.00213
第三阶段	0.00112	0.00120	0.00166	0.00198

需要注意的是，第一阶段新增多震级数据导致分布变化，后续学习率调整后恢复优化

多阶段训练展现出良好的收敛特性，各指标均呈现逐步优化的趋势，最终模型在训练集和验证集上都达到了优异的性能表现。

1.5 5. 训练配置详解

1.5.1 共同配置参数

以下是各训练阶段共享的基础配置参数：

```
{
    "epoch_train": 30000,      // 训练周期数
    "use_model": "LSTM",      // 使用 LSTM 模型
    "kernel_units": 32,      // LSTM 单元数
    "use_power_loss": true,    // 启用功率对数损失
    "use_points": 8000,      // 使用的时间点数量
    "RESTART_AFTER_N_CYCLES": 300, // 学习率重启周期
    "use_scale": true,        // 启用特征缩放
    "sample_rate": 2000,      // 采样率
    "time_clipped_s": 4.0,    // 时间窗口长度
    "activation": "tanh",      // LSTM 激活函数
    "dataset_type": "Alias",   // 数据集类型
    "feature_range": [-1, 1]   // 特征缩放范围
}
```

这些参数构成了训练的基础框架，确保各阶段训练的一致性和可比性，同时通过特定参数的调整实现差异化的训练策略。

1.5.2 优化器与损失函数

模型采用 Adam 优化器和复合损失函数：

```
# 优化器配置
optimizer = tf.keras.optimizers.Adam(learning_rate=config['learning_rate'])

# 损失函数配置
model.compile(
    optimizer=optimizer,
    loss='mse', # 均方误差作为主损失函数
    metrics=[power_log_loss] # 添加功率对数损失作为评估指标
)
```

主损失函数使用均方误差(MSE)，同时使用功率对数损失(power_log_loss)作为辅助评估指标，这种组合有助于增强模型在频域特性上的拟合能力。

```
def power_log_loss(y_true, y_pred, group_points=4000):
    # ===== 1) 以 group_points 为单位拆分序列 =====
    # 假设 seq_num 能被 group_points 整除
    if len(y_true.shape) == 2:
        # shape: (batch_size * group_points, feature_num) -> (batch_size, group_points, feature_num)
        y_true = tf.reshape(y_true, [-1, group_points, y_true.shape[-1]])
        y_pred = tf.reshape(y_pred, [-1, group_points, y_pred.shape[-1]])

    power_true = tf.reduce_sum(tf.abs(y_true), axis=1)
```

```

power_pred = tf.reduce_sum(tf.abs(y_pred), axis=1)

# 对能量取对数, 确保偏高和偏低的能量损失对称
# 对数不需要归一化, 因为本身对数包含了相除的操作
# 能量对数 shape: (batch_size, feature_num)
power_true_log = tf.math.log(power_true + 1e-8)
power_pred_log = tf.math.log(power_pred + 1e-8)

# 计算能量对数的均方误差
loss_power_log = tf.abs(power_true_log - power_pred_log)

# 最后加和平均得到最终损失
loss_power_log_avr = tf.reduce_mean(loss_power_log)

return loss_power_log_avr

```

功率对数损失函数通过计算波形能量的对数差异, 提供了一种频域层面的评估指标。该函数特别关注信号的能量分布特性, 而不仅仅是时域上的点对点差异, 这对于处理非线性失真特别有效。

1.5.3 学习率调度策略

实现余弦退火与周期性重启的学习率调度:

```

# 学习率调度实现
def lr_schedule(epoch, lr, decay_steps=100, restart_steps=300):
    """余弦退火学习率调度, 支持周期性重启"""
    # 计算当前周期内的步数
    step_in_cycle = epoch % restart_steps

    # 计算余弦衰减系数
    cosine_decay = 0.5 * (1 + np.cos(np.pi * step_in_cycle / decay_steps))

    # 如果超过衰减步数, 使用最小学习率
    if step_in_cycle >= decay_steps:
        return lr * 1e-3 # 最小学习率

    # 否则应用余弦衰减
    return lr * cosine_decay

# 学习率调度器回调
lr_scheduler = tf.keras.callbacks.LearningRateScheduler(
    lambda epoch, lr: lr_schedule(
        epoch,
        lr,
        decay_steps=config['auto_lr_decay_steps'],
        restart_steps=config['RESTART_AFTER_N_CYCLES']
    )
)

```

学习率调度采用余弦退火策略, 配合周期性重启机制, 能够有效避免模型陷入局部最优解, 同时根据训练阶段的不同需求调整衰减速度。

1.6 6. 模型参数继承机制

1.6.1 权重文件管理

各阶段训练的权重文件有统一的命名和存储规则:

```
projects/LSTMu32a1_rs300_m5/data/LSTMu32a1_rs300_m5_best_weights.h5
projects/LSTMu32a1_rs300_ex/data/LSTMu32a1_rs300_ex_best_weights.h5
projects/LSTMu32a1_rs300_ex2/data/LSTMu32a1_rs300_ex2_best_weights.h5
projects/LSTMu32a1_rs300_ex3/data/LSTMu32a1_rs300_ex3_best_weights.h5
```

这种规范化的文件结构便于权重文件的管理和继承，确保多阶段训练的连贯性。

1.6.2 配置继承实现

通过 `base_project` 参数指定继承关系：

```
{
    "base_project": "LSTMu32a1_rs300_ex2",
    // 其他配置参数...
}
```

此机制允许新阶段训练直接继承前一阶段的模型配置，简化了配置过程并确保一致性。

1.7 7. 实验结果与评估

1.7.1 模型参数量统计

```
# LSTM 模型参数量统计
参数量 = 4 * (input_dim + units) * units + 4 * units
当 input_dim=1, units=32 时:
= 4*(1+32)*32 + 4*32 = 4224 + 128 = 4352
dense_params = 1056 # 全连接层
output_params = 33 # 输出层
total_params = 5441 # 总参数量
```

LSTM 模型总共包含约 5,441 个参数，其中 LSTM 层占据最大比例，参数量适中，既能保证模型的表达能力，又不会造成过拟合。

1.7.2 训练过程分析

模型训练呈现出良好的收敛特性：

```
# 第三阶段 vs 第二阶段性能改进
improvement = (0.00118 - 0.00112) / 0.00118 * 100 # 约5.6%的损失值降低
```

最终的第三阶段相比第二阶段，损失值降低了约 5.6%，验证了多阶段训练策略的有效性。特别是单震级预训练模型在简化数据上取得的良好性能，为后续的多震级训练奠定了坚实基础。